Notes

Software Products

Generic products

- Standalone systems
- Marketed and sold to any customer
- Specification: What software should do is owned by the software developer and decisions made by the developer

Customized products

- Commissioned by a specific customer (their own need)
- Specification: What software should do is owned by the customer and they make decisions about the changes

Good software attributes

- Maintainability
- Dependability
- Security
- Efficiency
- Acceptability

Software engineering

- Discipline
- Concerned with all aspects of software (from early stages of spec. to maintaining)
- Using appropriate theories and methods to solve problems
- Technical process of dev., proj. management, dev. tools, dev. methods
- Cheaper in the long run

Fundamentals

- Systems should be developed using a development process
- Dependability and performance are important
- Understand and manage spec. and requirements
- Using software for reuse

Web-based

- Complex distributed systems
- Software reuse
- Incremental and agile development
- Service-oriented systems
- Rich interfaces

Software process

Activities

- Specification
- Development
- Validation
- Evolution

Common issues

- Heterogeneity: Distribution on diff. systems and platforms
- Business and social change
- Security and trust
- Scale: From embedded systems or wearable devices to cloud-based, web-based solutions

Application types

- Stand-alone application
  - On local computer
  - Network is not needed
- Interactive transaction-based applications
  - Execution on remote computer
- Embedded control systems
  - Control and managing hardware devices
- Batch processing systems
  - Process data in large batches
- Entertainment systems
  - Primarily for personal use
- Systems for modelling and simulation
  - Model physical processes or situations
- Data collection systems
- Systems of systems
  - Systems are composed

Architectural design

- How the software system should be organized and designing the overall structure
- Link between design and requirements
- Identifies the main structural components
- Output: Architectural model
- Architecture includes other systems, programs and program components (complex structure)
- Pros:
  - Stakeholder communication
  - System analysis
  - Large-scale reuse
- Representations
  - Informal block diagrams (entities and relationships)
  - Very abstract, but it is understandable for the stakeholders

Design decisions

- Is it a generic app?

- How will it be distributed?
- What arch. patterns or styles should be used?
- What will be the approach for the structuring?
- What strategy will control the operation of the components?
- How will the components be decomposed?
- What organization will be the best for the non-func. requirements?
- How should the documentation happen?

Architecture reuse

- For similar solutions (like product lines)
- Patterns or styles

Architecture characteristics

- Performance
- Security
- Safety
- Availability
- Maintainability

Architectural views

- Logical view
  - Key abstractions in the systems (objects or object classes)
- Process view
  - Composing of interacting processes in run-time
- Development view
  - How the software is decomposed during development
- Physical view
  - Which hardware or software components are distributed across the processors
- Representation by UML
  - Or by architectural description languages (rarely used)

Architectural patterns

- Representing, sharing and reusing knowledge
- Good practice
- Tested in diff. environments
- Represented by tabular and graphical descriptions

Model-View-Controller (MVC)

- Separates presentation and interaction from the system data
- Structure of logical components
- Generic solution for dynamic data representation

Layered architecture

- Interfacing of sub-systems
- System is set of layers with services
- Supports incremental development on different layers
- The interface changes only affect the adjacent layer

Repository architecture

- Data exchange between sub-systems through a central database
- Mostly for large data sharing

Client-server architecture

- Distributed system model
- Data and processing are distributed across a range of components
- Stand-alone servers with specific services and set of clients that are calling them through network

Pipe and filter architecture

- Functional transformations process
- Sequential transformations for data processing

Application architecture

- Reflects the application requirements and follows organizational needs
- Starting point for the architectural design
- A design checklist
- Development team organizing ways
- Common vocabulary

Application types

- Data processing application
  - Data driven application
  - Process data in batches
  - No need for explicit user intervention
- Transaction processing application
  - Data-centered application
  - User requests and update information in a database
  - Asynchronous requests for a service that is processed by a transaction manager
  - Information Systems (also web-based)
    - Client-server, layered model
- Event processing systems
  - Action from environmental events
- Language processing systems
  - User intentions in a formal language and interpreted by the system
  - Natural or artificial language as input
  - Compiler
    - Lexical analyzer, symbol table, syntax analyzer, syntax tree, semantic analyzer, code generator
    - Pipe and filter or repository

Software reuse

- Composing existing components
- Create better software with specific design processes to support reuse
- Categories:
  - System reuse
  - Application reuse
  - Component reuse

    - Object and function reuse
- Benefits
    - Accelerated development
    - Effective use of specialties
    - Increased dependability
    - Lower development costs
    - Reduced process risk
    - Standards compliance
- Problems
    - Creating, maintaining and using component library
    - Finding, understanding and adapting reusable components
    - Increased maintenance costs
    - Lack of tool support
    - Not-invented-here syndrome
- Planning factors
    - Development schedule
    - Expected software lifetime
    - Background, skills, experience
    - Non-functional requirements
    - Application domain
    - Execution platform

Reuse approaches

- Application frameworks
- Application system integration
- Architectural patterns
- Aspect-oriented software development
- Component-based software engineering
- Configurable application systems
- Design patterns
- ERP systems
- Legacy system wrapping
- Model-driven engineering
- Program generators
- Program libraries
- Service-oriented systems
- Software product lines
- Systems of systems

Application system reuse

- Can be adapted for different customers without changing the source code
- Application systems have generic features
- Can be reused in different environments
- Built-in configuration mechanisms for tailoring
- Benefits:
    - Rapid deployment
    - Easier to judge the functionality is good or not
    - Avoid some development risk (tested and used software)
    - Focusing on core activity
- Problems:

        - Requirements must be adapted
        - Assumptions
        - Difficult to find the correct application system
        - Lack of expertise to support system development

Configurable application systems

- Generic application systems for a particular business type, activity
- Domain-specific systems for business function
- Generic solution, standard processes
- Development focuses on configuration
- Vendor is responsible for maintenance

Application framework

- Large entities that can be reused
- Sub-system design made up to reuse
- Collection of abstract and concrete classes and interfaces
- Need to add some components by adding some implementation
- Web application frameworks exist for dynamic web pages
    - MVC pattern commonly used
    - Features: Security, dynamic web pages, db support, session management, user interaction
- Generic solutions that can be extended to specific tasks
    - Add concrete classes (inherit operations from abstract)
    - Adding middleware methods that are called by the system
- Problem: The complexity is hard to understand
- Rely on object-oriented features (polymorphism)
- Providing technical support

Software product lines

- Application families
- Application with generic functionality that can be adapted and configured for a specific context
- Set of applications with common architecture and shared components
- Adaption:
    - Component and system configuration
    - Adding new components
    - Selectin existing components from a library
    - Modify components to meet new requirements
- Structure:
    - Base:
        - Core components
        - Not usually modified
    - Configurable components:
        - Can be specialized with configuration with changing their code
    - Specialized, domain specific components
- Providing domain-specific support
- Family application, usually owned by the same organization
- Specialization:
    - Platform specialization
    - Environment specialization
    - Functional specialization
    - Process specialization

Instance development

- Select stakeholder requirements
- Choose closest-fit family members
- Re-negotiate requirements
- Adapt existing system
- Deliver new family member

Product line configuration

Design time configuration

- Modify the software during development with adaptation or selection

Deployment time configuration

- Configurate by a customer or consultant
- Configure during development
    - Component selection (select modules for the required functionality)
    - Workflow and rule definition (applying rules by entering or generation)
    - Parameter definition

ERP systems

- Generic system that supports common business processes (ordering, invoicing, manufacturing)
- For large companies
- Generic core adapted with modules
    - Each module is a business process
- Common database
- Configuration
    - Select required functionality
    - Establishing data model
    - Defining business riles
    - Defining expected external interactions with the system
    - Designing input forms and output reports
    - Designing new business processes
    - Setting parameters for underlying platform

Integrated application systems

- Application that includes two or more application system products

Component-based development (CBSE)

- Software development relies on reuse entities of software components
- Support effective reuse
- Components are abstract entities and can be stand-alone
- Components are independent, follow standards, connected by middleware
- Implementations are hidden
- Communication through interfaces
- Components are replaceable

Components

- Provides services
- Not important the programming language or the executing environment
- Independent executable
- Interface is published and all interactions are through the published interface
- The inner state and attributes cannot be seen
- Environment independent / independent deployment
- In/Out interface (well-defined)
- Composable, composition unit
- Explicit environment dependencies
- Documented
- Standardized
- Goal: reusing
- Interface contract
    - Functional aspect
    - Non-functional aspects
    - Pre and Post conditions

Component interface

- Provides interface
    - Services that are provides to other components
- Requires interface
    - Defines the services that are needed for the proper execution of the component

Conditions

- Pre-condition
    - Condition at the start of the calling
    - Can specify on inputs or initial state
- Post-condition
    - Condition at the end of the execution
    - Can specify on outputs or final state
- The inner state just can be observed from outside

Quality attributes

- Performance
- Dependencies
- Documentation about the attributes

Component meta-model

- Component and its connections
- The model of the model
- Conception about the future physical representation
- The interface (provides, requires) that is published

Component model

- Definition of standards for component impl., documentation and deployment
- Specifies how interface should be defined
- Should include the interface definition

- Elements:
    - Interfaces
    - Usage
    - Deployment
- Component platform: the running infrastructure of the model
    - Interactions between components
    - Component life cycle
    - Extra functional characteristics
- Component framework:
    - Ensuring service using or platform reaching

CBSE Processes

- Processes that support component-based engineering
- Development for reuse
    - Component development
    - Generalized solution
    - Or we can generalize an existing application
        - Remove app specific methods
        - Generalize names
        - Consistent exception handling
        - Configuration and adaptation
        - Example: wrapping legacy system
    - Should: hide inner state, independent as possible, publish exceptions through interface, reflect stable domain abstractions
        - Trade-off between reusability and usability
        - Should expose exceptions but not all
    - Higher development cost but it is cost efficient after reusing
    - Less space efficient and has longer execution time
- Development with reuse
    - Trade-off between ideal requirements and provided services
    - Searching components, modify requirements, search again and compose to create a system
    - Component identification: search, selection and validation
- Supporting processes:
    - Component acquisition: acquiring components for reuse or development
    - Component management: cataloged, stored and available component store
        - Repository the is discoverable
        - Maintaining information about the components
        - Keeping track for different versions of the component
    - Component certification: checking and certifying that it meets the specifications

Component composition

- Assembling components to create a system
- Integrate components with each other
- Normally we need glue code
    - Resolve interface incompatibility
        - Parameter incompatibility: operation have the same name but are of different types
        - Operation incompatibility: name of operations in the composed interfaces are different
        - Operation incompleteness: the provides interface of one component is a subset of the requires interface of another
- Types:

        - Sequential composition
            - Composed components are executed in sequence
            - Composing the provides interfaces of each component
        - Hierarchical composition
            - One component calls the services of another
            - The provides interface of one component is composed with the required interface of another
        - Additive composition
            - The interfaces of two components are put together to create a new
            - Provides and requires interfaces are combinations
- Trade-offs:
    - Conflicts between functional and non-functional requirements
    - Conflicts between rapid delivery and system evolution

Adaptor components

- Solve the component incompatibility

COTS

- Read-to-use components
- Cost-effective, professional
- We only know the functionalities (not what inside it)
- Not fully good for us (not developed for us)
    - It's a general solution
- It has specific architectural requirements
- No full control over the component
- Need to adapt to use the interface

Architectural differences

- Not all the components are compatible with each other
- Too many assumptions -> they cause problems
    - Provides assumptions: he provided component service descriptions
    - Required assumptions: the required services that are needed for the proper work

Wrapper

- Wrapping the component into an alternative abstraction

Bridge

- It transforms the first component required assumptions into the second one
- It is independent from the components

Negotiator

- Like the bridge, but it works in run-time
- If the system has a transformator (bridges) for D1->D2 and a D2->D3 that its dynamically can create a transformator for D1->D3

Techniques to explore differences

- Create conformance map
    - Describes the component attributes that can be observed from outside
    - It gives a mapping between the used and our symbol system connections

- Create semantic map
    - Describes the specification differences and similarities between the two components
    - Tries to model the two interfaces

Design with searching

- Search for the most fitted component
- Check the integrability into the architecture
- Exploration path
    - With all the found entity
    - Mainly contains the not-adaptable arch. differences but it contains the impact of the fix or ignore
    - Check one main path and some secondary
- Model problem
    - For scoring and quality guaranties
    - The implementation restrictions
- Model solution: the component that is fitting prototype by the design pattern

Model problem

- Step 1: The architecture and the engineer identify an initial state
- Step 2: The architecture and the engineer define the pre-conditions
- Step 3: The architecture and the engineer define the implementation restrictions
- Step 4: The engineer creates a problem solution
- Step 5: The engineer identifies the post-conditions
- Step 6: The architecture evaluates the model solution by the post-conditions

KobrA

History

- Top-Down solutions
    - Split the system into sub-systems, components, etc.
    - Specify, design and implement each component
    - Integrate the components
    - Problem with top-down
        - The created components, sub-systems only compatible with the other components in the system
- Component-based solutions
    - Bottom-up
    - Re-use existing components
    - Basic components
    - Two main parts: component development and application development (from components)
    - Component development:
        - Re-usable component with the given functionality
        - It must be integrable
    - Application development:
        - Re-using components and integrate them
        - Decomposing huge parts into smaller components
- We need a effective development model (analysis, design, implementation, verification, documentation)
    - The model is a collection of methods, methodologies
    - Helps in the development during the app life cycle

Model

- Analysis, design, programming, verification, documentation
    - UML is a tool for analysis and design
- Goal: create an abstract form for the structure of implementation independent
- Key points:
    - Separate problems
    - Model based development
    - Component using
    - OOP
- It has a 3D model
    - Composition/Decomposition
        - Decomposition goal: re-usable components
        - Embodiment during decomposition
            - Identify and specify the sub-systems
            - The specification and implementation are separated
            - The specification is very abstract (client can understand it)
            - Process: Executable program creation from UML through the concrete representations
        - Composing the decomposed elements (integration)
        - Validation:
            - Compare the implemented solution with the abstract model
            - Not at the end of the decomposing (we can validate after each decomposing process)
    - Abstraction/Concretization
        - From the understandable to the computer language
    - Generic/Specialized
        - For product-line application
- Spiral model
    - We can identify new elements during decomposition -> embodiment, decomposition, composition -> validation again
    - The steps can be repeated at different abstraction levels

Environment map

- The starting point of development, the specification of the application
- The specification is inherited from the requirements
    - The requirements are collected from the users (use case diagrams)
- Environment describer data
    - It needs to represent the environmental requirements
- Environment definer descriptive elements are the environment map
- The KobrA handles it as a component
- The result of the map: the required interface of the system
- The definition of the environment is the first step
    - What are the supported operations?
    - What is the system input?
    - Who/What will interact with the system?
    - What is needed from the environment?
    - Can be defined by use case diagrams

Usage model

- Specify the connection between the user and the system

- Use-case diagrams
    - Interactions between system components
    - Interactions between the system and its borders
    - Show the users of the system and the connections between them
    - Each diagram is an abstract function
- Diagrams should be described with descriptions

Enterprise or Business Process model

- Describe the business environment of the system
- Defines the definitions and keywords, meanings

Structural model

- Defines the structure of the entities (that are outside of the system that need to be implemented but have impact on the system)
- Contains the input and the output (that are used in further processes)
- UML class or object diagram

Interaction or Behavioral model

- UML activity diagram or interaction diagram
- Abstraction of technical details from the usage model
- Recursive specification process (detailed result step-by-step)
- Behavioral model
    - The impact of system users on the system border
- Interaction model
    - The activity of the system users (the user is the main element)

Component specification

- Documents that define the components
    - Informal description
    - UML
    - Formal tools: OCL or QoS Modelling Language
- Goal: understand the behavior and make usable
    - Describe the requires and provides interfaces

Structural model

- The structure of the component and the connected other components
- Defines the operations and attributes of the component
- Defines the restrictions of the connections to other components
- Focus on the system (not the environment)
- Shows the decomposition possibilities
- UML class and object diagrams

Functional model

- The provides and requires operations (functionalities)
- Operation specification (defines the impacts that are observable from outside)
    - It's a table with Assumes and Result clauses (pre- and post-conditions of the operations) -> answers the what questions but no the how.

Behavioral model

- Defines the behavior of the component
- Pre- and Post-conditions
- UML state diagram
- The encapsulation of the functionality and the data
    - An object has states and state transitions
- Uses the functional model Assumes and Result clauses
- If a component does not have state, no need for this model
- These states are only observable from outside
    - Observe the result of an event
- A transition can have a guard condition

Additional docs.

- Quality ensuring plan
- Full documentation of the component
- Decision model

Component implementation

- The collection of the docs that describe the method of the implementation
- High level components can be created with integration of low-level components
- Contains:
    - sub-component specifications and their requires and provides interface
    - own implementation (private implementation)
    - the component structure model (inner structure)
    - specification of embodied algorithms
    - interactions with other components (UML sequence or interaction diagram)

Structural model

- UML class and object diagrams
- The class and its connections inside the component
- The inner architecture of the component
- It is the refinement of the specification's structural model
    - New elements can appear
    - In some cases, the document more specific than in the specification, in some cases more abstract

Behavioral or Algorithmic model

- UML activity and state diagrams
- Algorithmic model
    - Detailed specifications about the operations
    - Activity diagram

Interaction model

- UML sequence and collaboration diagram
- Shows the process of the control
- How the components are collaborated to execute a process

Component embodiment

- The component becomes an executable, deployable entity from the abstract specification
- One high level component
    - Decomposed into sub-components through decomposition-concretization layer (spiral) -> we got binary at the and
- Some steps are automatizable (model transformation is hard -> because there is a huge gap between the model and the semantic program)
    - We can use implementation near models/diagrams like UML
    - We need some implementation decision

SORT

- Systematic Object-Oriented Refinement and Translation
- Separate the refinement from compilation
- NOF
    - Normal Object Form
    - An implementation profile
    - Minimize the gap between object-oriented models and object-oriented languages
    - UML stereotypes (additional restrictions)
    - Supports the model-based design (OOP)
    - Translate the UML to the OOP core definitions
    - Automatic transformation to OOP languages from graphical representation
- Refinement
    - Same relations on different abstraction levels
- Compilation
    - Relation between the two descriptions on the same abstraction level
- Realization details -> refinement -> translation (into source code) -> automatic transformation (into physical component)
- Advantages
    - The sequence of the smaller steps is more understandable
    - The graphical model implementation process is splitted into refinement and compilation steps
    - The problems will be separated
        - One problem at a time
    - Easier to find the refinement places or to identify the similarities

Component reuse

- Not fully fulfill the specification
- Solution: add adapter component to translate functionality between two components
- Semantic translation between different entities (like glue code)
- If there is not any component for reuse, we need to create a new one
    - Decomposing and finding components to reuse for the decomposed sub-components
    - Integrate the sub-components together and we get a new component

Component integration

- Easiest way: the existing component specification is equivalent to our specification
- Conformance map
    - Describes the attributes that are observed from outside
    - The connection between the symbols in our system and the component

        - Only available option, when the reused component documentation we can get the correct information (about structure, behavior and functionalities)
            - The information is often distributed and complex
        - If we can create the conformance map, we can check that the reused component is good for our goals (by comparing the interfaces)
- Semantic map
    - Comparing the component differences and similarities (between the interfaces) -> wrapper component specification (adapter)

Component construction and deployment

- The last steps
- Deployment to the target platform

Product Family concepts

- The reusing question is on the architecture level
- The product family application is a generic component system
- Decision model
    - We need to check the differences and similarities between the family applications
    - The model defines three things to a question
        - The question
        - The possible answers
        - The place or person where the decision will be made
    - During the development, decision models must be made for the models
        - environment map
        - component specification
        - component implementation

Documentation and quality ensurance plans

- Quality ensurance requirements
    - Defines the definition of quality
    - Defines the quality aspects and quality levels
- Documentation is part of the component specification and specification refinement
    - Provides deep understanding and help for the use

Composition

- Assemble software construction into a bigger entity

Composition views

- Programming view
    - The programmer assembles the code parts to create the application
    - Composition entities: methods, functions, classes, aspects
    - Composition mechanism: sequence of the programming language
    - Reusability is not an important matter
- Construction view
    - Assemble components with connectors to create the application
    - Systematic construction
    - Composition entities: components and connectors
    - Composition mechanism: script languages
    - Design result: component and their connections (architecture)

        - Not managing the existing entities
- Component based development (CBD) view
    - Extends the construction view with reusing
    - Systematic reuse
    - Composition entities:
        - Objects: Have provides interface but not have requires interface
            - Objects are connected through method callings
        - Arch. units: Input and output ports
            - Arch. units are connected through compatible ports
        - Encapsulated components: Only a provides interface (encapsulate operations)
            - Encapsulated components cannot be connected without a exogen composition connector
    - Composition mechanism:
        - Containment:
            - The behavior units are encapsulated into the composite entity
            - And the connected component behaviors define the composite behavior
            - Composition (it handles the composed object life cycle) and aggregation (the composed object life cycle is independent)
        - Extension:
            - The composite behavior is the extension of the composed objects
            - Like multiple inheritance
        - Connection:
            - The composite behavior is the connection of the behaviors of the composed objects
            - The composite units are calling each other (direct or indirect)
                - Direct: the objects are calling each other's methods (delegation)
                - Indirect: Message through a connector
        - Coordination:
            - The composite behavior is the behavior of the coordinated composed entities behaviors
            - The coordinator is responsible for the coordination that is connected to the composed entities
            - The composed entities are not connected to each other

Exogen composition

- The components are fully encapsulated
- Types:
    - Atomic components
    - Composition components
    - Composite component

Algebraic composition mechanisms

- Only the exogen coordination is algebraic
- The data coordination results process set and the orchestration results work processes (non-algebraic)
- Composition operators
    - Automatic glue code from defined composition operators
    - Containment does not have composition operators
    - Extension and Connection has composition operators in a few cases

Incremental composition

- Method to identify components and create composition
- The requirements should be translated into partial component architecture
- The partial component architecture should be extended with new and new requirements
- The extended architecture solved the new requirements and all previously
- Used composition mechanism: exogen coordination
- Composition connectors:
    - SEQ: sequence
    - PIPE: like sequence with data transfer
    - SEL: selector
    - LOOP: loop
    - GUARD: control condition
- Supports incremental design
- States:
    - Open composition connector: open composition and open interface
    - Closed composition connector: closed composition and closed interface
- A open composition can be closed, if all the composed entities are closed
- Steps:
    - Step 1: identify actions and choose components
        - Choose existing or new component
    - Step 2: identify control flows and choose composition connector
    - Step 3: create partial architecture
    - Step 4: compose partial architecture with the previously constructure architecture
        - Composition is not always possible because of the lack of composition point
        - We can delay the composition until a correct composition point
            - Invalid specification can cause no existence of correct composition point
    - Step 5: finalize full architecture
        - After all the processed requirements
        - We can refine the open composition points
            - Composing, merge connectors
            - Optimalization
        - Close all opened composition points

Service-oriented software engineering

Web services

- More general notion of a service
- Independent application
- The range of services is from different organizations
- Reusable components
- No requires interface
- Platform and impl. language independent
- Benefits:
    - Can be provided by any service provider (inside or outside of the organizations)
    - The provider makes information about the service public
    - The users can pay for services according to their use
    - Application can be smaller (the computation on the remote service)
- Service-based applications may be constructed by linking services from various providers

Service-oriented architecture

- Developing distributed systems (components are stand-alone services)
- Services may be executed on different computers
- Standard protocols support service communication and information exchange
- Benefits:
    - Can be provided locally or outsourced to external providers
    - Language independent
    - Legacy system can be preserved
- Standards
    - SOAP: message exchange standard for service communication
    - WSDL: language to define service interface and its bindings
    - WS-BPEL: standard for workflow language used to define service composition
        - what operations the service supports and the format of the messages that are sent and received by a service
        - how the service is accessed
        - where the service is located

RESTful web services

- Arch. style based on transferring representations of resources from a server to a client
    - Resource is simply a data element
    - Resource operations: Create (POST), Read (GET), Update (PUT), Delete (DELETE)
    - Data is exposed and is accessed using its URL
- Simpler than SOAP/WSDL
- Involve lower overhead
- Disadvantages:
    - Difficult to design when the service is complex, and the resource is not simple
    - No standard for interface descriptions (only informal documentation)
    - Need to implement own infrastructure for monitoring and managing quality

Service engineering

- Process of developing services for reuse in a service-oriented system
- Service must be robust, reliable, reusable and documented
    - Documentation: can be discovered and understood
- Stages:
    - Service candidate identification
        - Define service requirements and identify possible services
        - Service should support business processes
    - Service design
        - Design the logical service interface and its implementation interfaces
    - Service implementation and deployment
        - Implement and test the service and make it available

Types of service

- Group 1:
    - Utility service: implement general functionality
    - Business service: associated with a specific business function
    - Coordination service: support composite processes
- Group 2:
    - Task-oriented service: associated with some activity

        - utility, business or coordination services
    - Entity oriented service: associated with a business entity
        - utility or business services

Catalog services

- Created by a supplier to show which goods can be ordered from them by other companies
- Requirements
    - Specific version for each client
    - Shall be downloadable
    - Specification and prices are comparable (up to 6 items)
    - Browsing and searching for possibilities
    - Functions with predicated delivery date
    - Virtual order reserve for 48 hours
- Non-functional requirements
    - Only restricted employee access
    - Offered prices and configurations are confidential for each organization

Service interface design

- Service operations and exchange messages
- Minimized service requests
- Services state information may have to be included in messages
- Stages:
    - Logical interface design: service requirements, defines the operation names and parameters (exceptions should also be defined)
    - Message design: for SOAP
        - Structure of input and output messages
    - Interface design: for REST
        - How the required operations map into REST operations and what resources are required

Service implementation and deployment

- Standard programming language or workflow language
- Test: created input message and check produced and expected response
- Publicizing the service and installing it

Legacy system services

- Service interface implementation to existing legacy systems
- Legacy systems: extensive functionality and reduced impl. cost

Service composition

- Existing services are composed and configured to create a composed service or application
- The basis is of a workflow
    - Sequences of activities (services)
    - Construction:
        - Formulate outline workflow
        - Discover services
        - Selecting possible services
        - Refine workflow
        - Create workflow program

        - Test completed service or application
- Testing:
    - Functional and non-functional requirements
    - Black box
    - Non-functional behavior is unpredictable
    - Testing is expensive
    - Difficult to check connection with other services